Jupyter Book Plugins

Luuk Fröling, Freek Pols

Contents

1. editor **1**
- 1.1 This page can be edited through a wizard 1
- 1.2 hello 2

2. General book plugins **4**
- 2.1 Add date via frontmatter 4
- 2.2 Add date via csv file 5
- 2.3 Gallery grid 6

3. PDF Support **8**
- 3.1 PDF support - custom directives 8
- 3.2 Iframe specific testing page 10

4. cool stuff **12**
- 4.1 Reads a csv file 12
- 4.2 Grasple and H5P Exercises 13

5. Typst support **15**
- 5.1 Typst support - raw typst 15
- 5.2 New typst element 16
- 5.3 Equations 17

6. gh-releases **19**
- 6.1 Using gh-release 19

7. GitHub actions **21**
- 7.1 Deploy Multi-Language MyST Books 21

8. Overleaf **24**
- 8.1 Using Jupyter Book with Overleaf, Visual Studio Code and GitHub Pages 24

9. Giscus **27**
- 9.1 Add Giscus to book 27

1. editor

1.1 This page can be edited through a wizard

this is some text!!

1.2 hello

1.2.1 Notes (written in the editor):

- the iframe still needs to be passed the file location
- the iframe does not replace all elements. this should be the case. this can be fixed using a plugin
- a lot of jb2 features (i think most things using :::{element}:::) are not supported. needs more testing.
- database does not work - if things go wrong **Go to f12 > application > Storage > IndexedDB > test_database > 'Delete database'

**- iframe shows itself to be edited, in the codeblock at the bottom of the page. Ideally we want to hide this part. (could be done in /app > parser > index.tsx > code which removes element so its not displayed. needs to be added back later to be committed to gh)

Formula here:

$$E = mc^2 \tag{1.1}$$

Do not remove or change this iframe:

2. General book plugins

2.1 Add date via frontmatter

adds updated date from frontmatter above.

requires custom css:

```
#skip-to-frontmatter {
  margin-bottom: 0;
  order: -2; /* Keep the original date first */
}

.article.content.article-grid {
  display: grid;
}

.updated-date-container {
  order: -1; /* Move it visually below the date */
}
```

For future reference : the order determines wether the abstract (order 0, default) is shown first or the updated date (manually set order −1).

2.2 Add date via csv file

use the dates.csv file located in the root of the repo.

2.3 Gallery grid

This plugin generates a responsive card-based gallery grid using data loaded from a YAML file. Each item in the YAML file becomes a card containing a title, image, description, and a link.

Use the directive in your Markdown, which will load data from `filename.yml`:

```
:::{gallery-generator} filename.yml
:::
```

If no filename is provided, it will default to `myst.yml`. The YAML file should define a list of items to appear in the grid:

```yaml
books:
- name: Jupyter Book 2 Workshop Template
  website: https://jupyter-book.github.io/workshop-template/
  repository: https://github.com/jupyter-book/workshop-template
  image: https://jupyterbook.org/stable/build/config-item-9ec68b1d-f72dcf20e0e
079cdebe5dc9e5fbd7418.svg
  description: A GitHub Template repository designed for use in Jupyter Book 2
and MyST workshops.
```

Example:

3. PDF Support

3.1 PDF support - custom directives

This text is responsible for the headers lining up in typst...

3.1.1 Adding plugins to your book

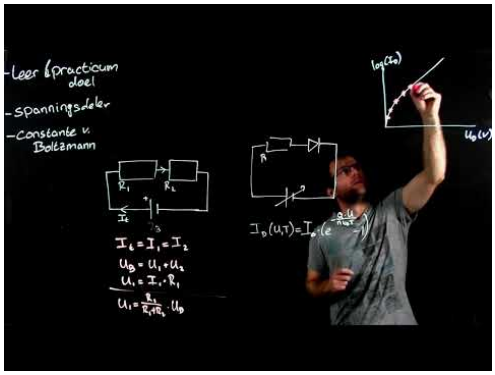
This page explains how to add a variety of plugins to your book. Each plugin can be linked individually in your myst.yml file.

⚠️ Important: Always include the CSS file (only once) to ensure the custom styles are applied.

Example myst.yml configuration:

```
site:
  template: book-theme
  options:
    style: https://github.com/TUD-JB-Templates/JB2_plugins/releases/download/
CSS/custom.css
```

3.1.2 iframe



(a) (b)

Figure 1: a great movie, and a great caption - Scan the QR code or click here to go to the video.

Use this link to add support for iframes when converting to pdf:

`https://github.com/TUD-JB-Templates/JB2_plugins/releases/download/iframe/iframe.mjs`

3.1.3 example

Here is an example

With some text.

Use this link to add support for the example directive when converting to pdf:

`https://github.com/TUD-JB-Templates/JB2_plugins/releases/download/example/example.mjs`

3.1.4 experiment

this is an experiment

with a description

Use this link to add support for the experiment directive when converting to pdf:

```
https://github.com/TUD-JB-Templates/JB2_plugins/releases/download/experiment/
experiment.mjs
```

3.1.5 intermezzo

this is an intermezzo

with a description

Use this link to add support for intermezzo's when converting to pdf:

```
https://github.com/TUD-JB-Templates/JB2_plugins/releases/download/intermezzo/
intermezzo.mjs
```

3.1.6 exercises

Exercise 1: my-exercise

1 + 1 = … ?

Solution to Exercise 1: my-exercise

2

Use this link to add support for exercises when converting to pdf:

```
https://github.com/TUD-JB-Templates/JB2_plugins/releases/download/exercise/
pdf_exercise.mjs
```

3.1.7 Custom element

To generate a custom element, including css

- style must have been included in css file.

3.2 Iframe specific testing page

The package that keeps on breaking

Figure 3.5:

Figure 3.4: a great movie, and a great caption - Scan the QR code or click here to go to the video.

4. cool stuff

4.1 Reads a csv file

with the intend to create a date

4.2 Grasple and H5P Exercises

Should embed grasple exercise:

Grasple 1: some exercise

Exercise 1: some exercise

Here an example of a normal exercise block.

Grasple 2

H5P exercise 1: Balloon in car

License: CC BY-NC-SA 4.0

5. Typst support

5.1 Typst support - raw typst

use

```
const generateTypstNode = (val) => {
  let result = {
    type: 'raw',
    lang: 'typst',
    typst: val
  }
  return result
}
```

to replace any node with a typst node.

Here is an example

With some text.

This is an aside

Sidenote with some text

5.2 New typst element

5.2.1 define element

Create new file, aside_style.typ in this example

```
// Custom typst element (https://typst.app/docs/reference/)

#let aside(title, body) = {

    // Title block
    block(
        // Outer block styled like a note block
        fill: rgb(244, 147, 147), // background - VS code automatically
changes this to rgba sometimes which does not work.
        stroke: (left: 1pt + red),  // red line on the left
        width: 100%,

        // add padding top and bottom
        inset: (x: 0.8em, y: 0.4em),
        above: 0.5em,               //distance top to next element above
        below: 0em,                 //distance bottom to next element
below

        strong(title)
    )
    block(
        // Inner block for the body of the aside
        fill: rgb(240, 240, 240),        // white background
        stroke: (left: 1pt + red),
        width: 100%,

        // add padding top and bottom
        inset: (x: 0.8em, y: 0.6em),
        above: 0em,
        below: 0.5em,

        body
    )
}
```

the content of the element follows the same style as a noteBlock.

5.2.2 Add new .typ file to template.yml

```yml
files:
  - template.typ
  - style.typ
  - aside_style.typ
  - LICENSE
  - Cover.PNG
```

5.2.3 add element via plugin

As we wanted to replace the aside element via a plugin, we inject the raw typst:

```
    rootChildren[i] = generateTypstNode(
        '#import "aside_style.typ": aside \n #aside("' + title + '", "' +
body.map(b => getText(b)).join("\n") + '")'
    );
```

Make sure to include the new file as well, before naming the element.

5.3 Equations

Some equation symbols like $\Rightarrow$ are not known in Typst and raise an error when converting to pdf. the plugin typst_eq.mjs replaces these symbols with their unicode equivalents.

6. gh-releases

6.1 Using gh-release

See also the MyST documentation as well as the GitHub documentation

6.1.1 Single file release

To release a single .mjs file:

- On the main page of the repository, go to About > Releases > Draft a new release.
- Add a tag to the release. Tags can be any name, but they're usually version numbers to indicate a release point.
- Add a title and a description. The published release will include a contributors section with an avatar list of all the @ mentioned users.
- Attach .mjs file in the dropbox.
- To notify users the release is not ready for production and may be unstable, select "This is a pre-release".
- Click "Publish release".

6.1.2 Multiple files release

6.1.2.1 Build plugin using esbuild

To build a plugin using esbuild:

- set-up folder structure with a src and dist folder and add all .mjs files to the src folder.
- add build.js file to the root of the plugin.

```
const esbuild = require('esbuild');

esbuild.build({
entryPoints: ['src/plugin.mjs'],
bundle: true,
outfile: 'dist/plugin.mjs',
platform: 'node',
format: 'esm',          // pure ESM output
external: ['fs', 'fs/promises', 'os'] // <- do NOT bundle these

}).catch(() => process.exit(1));
```

- In build.js, add the entryPoints, and the output file (outfile).
- run node build.js

6.1.2.2 release

Same steps as for single file release using the output file (dist/plugin.mjs).

7. GitHub actions

7.1 Deploy Multi-Language MyST Books

A GitHub action can be used to automate building and deploying multiple book translations (e.g. English and Dutch) to GitHub Pages. It's designed for repositories where each language has its own content folder, for example `content/en/` and `content/nl/`.

When you push to the main branch, this workflow:

1. Builds each language version separatly.
2. Places each build under its respective folder (e.g. `/en/`, `/nl/`).
3. Optionally redirects the site root (`/`) to a primary language (default: English).
4. Deploys the final site to GitHub Pages.

This workflow is compatible with any multilingual setup. You can add or duplicate language-specific build steps (like the English and Dutch examples) for more languages.

7.1.1 Overview of Repository Structure

Your repository should include langage-specific content directiories, such as:

```
content/
  ├──en
  │   ├── index.md
  │   └── ...
  ├── nl
  │   ├── index.md
  │   └── ...
  ├── figures
  ├── toc_en.yml
  ├── toc_nl.yml
  └── myst.yml
```

Each language version should have its own table of contents file (`toc_en.yml`, `toc_nl.yml`) located in the root of the project. The GitHub action will replace the toc file for each language during the build process.

> **Tip**
>
> If you place the `figures` folder at the same level as the language folders (rather than inside them), you can keep figure references consistent across all languages. This way, each translated book can use the same image paths without duplication.

7.1.2 Usage

1. create the workflow file `.github/workflows/deploy-multilang.yml`.
2. Copy the GitHub Action YAMl (TODO: Link).
3. Create a front page (for example content/index.md) that lets readers pick their preferred language. Each button should link to the root page of that language's build, which follows the pattern `https://{username}.github.io/{repository}/{language}`:

```
{button}`Dutch <https://{username}.github.io/{repository}/nl>`
{button}`English <https://{username}.github.io/{repository}/en>`
```

An example has been implemented here

> **Tip**
>
> you can customize the language list or behavior by editing the "prepare build folders", "Build" sections, and the "Root behavior" logic.

7.1.3 Workflow breakdown

Checkout and setup:

- Checks out the repository using `actions/checkout@v4`.
- Sets up Node.js using `actions/setup-node@v4`.
- Installs dependencies with `npm install -g mystmd js-yaml yq`.

Building each language:

- Each language build modigies `myst.yml` with the appropriate configuration.
- Runs `myst build --html --ci`.
- Copies the resulting files into `_build/<language>`.

Organizing output:

- After both builds, files are collected under `site/`:

```
site/
  ├──en/
  ├── nl/
  └── index.html (redirect or root behavior file)
```

Root behavior:

- Based on `BEHAVIOR_PRIMARY`, the workflow determines how to handle the root folder (`/`):
  - redirect creates HTML redirect
  - copy copies `/en/` to `/`
  - move moves `/en/` to `/` and removes `/en/`

Deployment:

- Uploads the built site using `actions/upload-pages-artifact@v3`.
- Deploys it to GitHub pages using `actions/deploy-pages@v4`.

8. Overleaf

8.1 Using Jupyter Book with Overleaf, Visual Studio Code and GitHub Pages

If you want your Overleaf LaTeX project to build automatically into a polished, interactive website and also integrate Python scripts and visualizations developed in VS Code, combining Jupyter Book with GitHub Pages provides a seamless way to turn your work into a continuously updated, professionally hosted site.

This guide walks you through connecting an Overleaf LaTeX project to GitHub and transforming it into a Jupyter Book–style site using MyST. It explains how to initialize the project, set up GitHub Pages deployment, and manage updates directly from Overleaf.

Prerequisites:

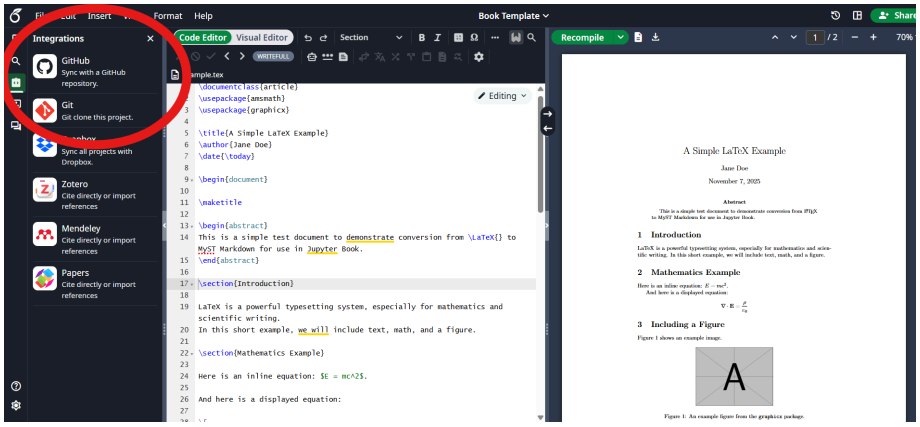
- A GitHub account
- An Overleaf account
- An existing Overleaf project
- Git or Github Desktop installed
- VS code installed
- Node.js available

8.1.1 Setting up

The steps below guide you through configuring your local environment, linking Overleaf to GitHub, and preparing the repository so it can be built and published with MyST and GitHub Pages. We will walk through the process in three parts: Overleaf, your local machine, and GitHub.

8.1.1.1 Overleaf

1. Open your Overleaf project.
2. Go to Menu > Integrations > GitHub.



3. Select `Create GitHub Repository`. Overleaf will generate a new repository and link your project to it.

You now have a GitHub repo containing your LaTeX files, ready to clone to your local machine.

8.1.1.2 Local machine

1. Open GitHub Desktop or use Git to clone the newly created repository to your computer.
2. If using GitHub Desktop, go to Clone repository > your repositories > {name of new repository}.
3. Open the cloned folder in Visual Studio Code.
4. Open the integrated terminal.

> **Tip**
>
> The next few steps will be similar to the Jupyter Book documentation, which you can refer to for more details about the process.

5. Install Jupyter Book

```
pip install "jupyter-book>=2.0.0a0"
```

6. Run the `init` command to ininitialise a `myst` project in the current directory. Do not run the `start` command (Jupyter Book will ask).

```
jupyter book init
```

7. Run the `init` command with the `--gh-pages` flag to setup a GitHub Action

```
jupyter book init --gh-pages
```

8. Commit and push all generated files to GitHub using Git or GitHub desktop.

This completes the one-time setup on your local machines, and future updates from Overleaf or your local machine will now trigger automatic site builds on GitHub Pages.

8.1.1.3 GitHub

The last step in the setup process is to give GitHub Actions permission to publish your site to GitHub Pages. In your repository settings, go to Settings > Pages, set the Source to "GitHub Actions," and save. Once enabled, every time you push to the repository, the action created earlier will automatically build and publish your Jupyter Book to the live site.

8.1.2 Usage

8.1.2.1 Changes using Overleaf

After making changes to your project in Overleaf, push the changes to GitHub by going to Integrations > GitHub > Push Overleaf changes to GitHub > sync. The gh-action will now rebuild your book, and update the website accordingly.

8.1.2.2 Changes using VS code

After making changes to your project in VS code, push the changes to GitHub by opening GitHub desktop, commit the files, and push to origin. Again, the gh-action will now rebuild your book, and update the website accordingly.

> **Note**
>
> Exercise 1: Try adding a Python plot of your data to your overleaf project.

8.1.3 Additional reading

- Jupyter Book Documentation

A guide to creating structured, book-style documentation using Markdown and MyST. Covers project initialization, configuration, file organization, and local builds.

- GitHub Pages

GitHub's built-in hosting service that publishes static websites directly from your repository.

- GitHub Actions

A workflow automation system that runs tasks, like building your Jupyter Book—whenever you push to the repository.

- Github Desktop
- Overleaf Integrations

Overleaf's built-in tools for linking your LaTeX projects to other services.

9. Giscus

9.1 Add Giscus to book

According the the Giscus page:

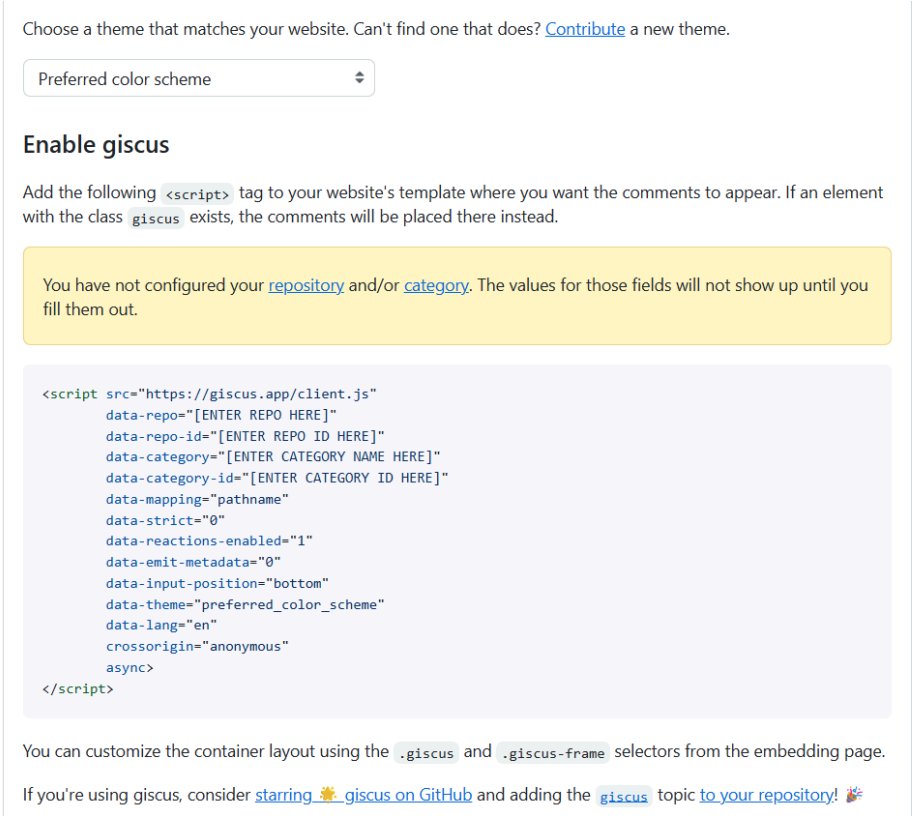
> A comments system powered by GitHub Discussions. Let visitors leave comments and reactions on your website via GitHub! Heavily inspired by utterances.

=== Setting up

We have to do 2 things, change the gh-action and add the giscus tag to our site.

9.1.0.1 Giscus tag

Follow the configurations on the website. This will generate a `<script>` tag which we will need.



Choose a theme that matches your website. Can't find one that does? Contribute a new theme.

Preferred color scheme

Enable giscus

Add the following `<script>` tag to your website's template where you want the comments to appear. If an element with the class `giscus` exists, the comments will be placed there instead.

You have not configured your repository and/or category. The values for those fields will not show up until you fill them out.

```
<script src="https://giscus.app/client.js"
        data-repo="[ENTER REPO HERE]"
        data-repo-id="[ENTER REPO ID HERE]"
        data-category="[ENTER CATEGORY NAME HERE]"
        data-category-id="[ENTER CATEGORY ID HERE]"
        data-mapping="pathname"
        data-strict="0"
        data-reactions-enabled="1"
        data-emit-metadata="0"
        data-input-position="bottom"
        data-theme="preferred_color_scheme"
        data-lang="en"
        crossorigin="anonymous"
        async>
</script>
```

You can customize the container layout using the `.giscus` and `.giscus-frame` selectors from the embedding page.

If you're using giscus, consider starring giscus on GitHub and adding the `giscus` topic to your repository!

Next, we want to

9.1.0.2 Github action

As Jupyter Book does not support custom js yet, we will have to adjust the gh-action to append our giscus tag to each webpage. By adding the following code to the deploy file, the gh-action will look for a static folder and append all js files found to our site.

```
- name: Insert static files
    run: |
        set -e

        HTML_DIR="_build/html"
        STATIC_SRC="static"
        STATIC_DEST="$HTML_DIR/static"

        echo "Searching for JS files in $STATIC_SRC..."
        js_files=$(find "$STATIC_SRC" -type f -name "*.js" | sort)
```

```
        js_count=$(echo "$js_files" | grep -c '.js' || true)

        if [ "$js_count" -eq 0 ]; then
        echo "⚠ No JS files found in $STATIC_SRC"
        exit 0
        fi

        echo "Found $js_count JS file(s)."
        echo "Copying JS files to $STATIC_DEST ..."
        mkdir -p "$STATIC_DEST"
        for js_file in $js_files; do
        cp --parents "$js_file" "$HTML_DIR"/
        echo "📁 Copied: $js_file → $HTML_DIR/$js_file"
        done
        echo "✅ Copy complete."

        echo ""
        echo "Building absolute script tag block..."
        script_tags=""
        for js_file in $js_files; do
        filename=$(basename "$js_file")
        script_tags="${script_tags}\n<script src=\"${BASE_URL}/static/
${filename}\"></script>"
        done

        echo ""
        echo "Searching for HTML files under $HTML_DIR..."
        html_files=$(find "$HTML_DIR" -type f -name "*.html" | sort)
        html_count=$(echo "$html_files" | grep -c '.html' || true)

        if [ "$html_count" -eq 0 ]; then
        echo "⚠ No HTML files found in $HTML_DIR"
        exit 1
        fi

        echo "Found $html_count HTML file(s) to update:"
        echo "Inserting <script> tags before </head> in each HTML file..."
        echo "------------------------------------------------------------"

        for file in $html_files; do
        echo " Updating: $file"
        sed -i "s|</head>|${script_tags}\n</head>|g" "$file"
        done

        echo ""
        echo "✅ Injection complete."
        echo "📊 Summary:"
        echo "  - JS files processed: $js_count"
        echo "  - HTML files updated: $html_count"
        echo "------------------------------------------------------------"
```

We can now write a js script to append our specific giscus tag, like in this example:

```
(function () {
  function insertGiscus() {
    // avoid duplicates
    if (document.getElementById("giscus_container")) return;


    const container = document.createElement("div");
    container.id = "giscus_container";

    // place it inside main.content or fallback to body set target by
classnames: article-grid subgrid-gap col-screen article content
    let target = document.getElementsByClassName("article-grid subgrid-gap
```

```
col-screen article content")[0];

    target.appendChild(container);

    const script = document.createElement("script");
    script.src = "https://giscus.app/client.js";
    script.setAttribute("data-repo", "TUD-JB-Templates/JB2_plugins");
    script.setAttribute("data-repo-id", "R_kgDOPrPv2Q");
    script.setAttribute("data-category", "Page comments");
    script.setAttribute("data-category-id", "DIC_kwDOPrPv2c4CxX_-");
    script.setAttribute("data-mapping", "pathname");
    script.setAttribute("data-strict", "0");
    script.setAttribute("data-reactions-enabled", "1");
    script.setAttribute("data-emit-metadata", "0");
    script.setAttribute("data-input-position", "top");
    script.setAttribute("data-theme", "preferred_color_scheme");
    script.setAttribute("data-lang", "en");
    script.crossOrigin = "anonymous";
    script.async = true;

    container.appendChild(script);
    console.log("💬 Giscus injected.");
  }

  // Run once at first load with 2 second delay to ensure DOM is ready
  document.addEventListener("DOMContentLoaded", () => {
    setTimeout(insertGiscus, 2000);
  });

})();
```

This will wait for 2 seconds for the page to load, before appending the comment section to the bottom of the page.

> **Warning**
>
> Make sure to change all atributes to correspond the generated tag